Axolotl 0.3.0 для Linux

Установка Axolotl и работа с компилятором командной строки axl

1. Что устанавливается

Axolotl: язык для JVM, который переводится в читаемый Java 21. Компилятор через NullAway проверяет, что значения, которые могут быть null, обработаны, и показывает каждую ошибку в строке файла `.axl`.

Установщик делает три вещи:

- ставит команду `axl` вместе с собственной средой Java 21, поэтому самому `axl` Java на компьютере не нужна. В среде есть все модули Java SE, включая AWT и Swing;
- добавляет `axl` в `PATH`;
- запускает `axl setup`, который копирует библиотеки Axolotl и плагины Gradle и Maven в локальный Maven-репозиторий, где их находят проекты Gradle и Maven.

2. Требования

- 64-битный Linux на x86 с glibc.
- Для `.deb`: Debian 12 или Ubuntu 22.04 и новее либо основанный на них дистрибутив, и `sudo`.
- Для архива tar.gz: любой такой Linux, root не нужен.
- Gradle и Maven, если вы ими пользуетесь, работают на собственном JDK 21: среда внутри `axl` служит только самому `axl`.

3. Установка

Debian и Ubuntu: пакет .deb

Скачайте `axolotl-0.3.0-linux-amd64.deb` со страницы https://github.com/succubus-uwu/axolotl/releases/tag/v0.3.0 и установите его через `apt`. Важно `./` в начале: без него `apt` ищет пакет с таким именем в репозиториях.

```
$ cd ~/Downloads
$ sudo apt install ./axolotl-0.3.0-linux-amd64.deb
```

Пакет ставится в `/opt/axolotl`, регистрирует `/usr/bin/axl` через `update-alternatives`, подтягивает библиотеки X11, ALSA и шрифтов, которые нужны среде Java, и выполняет `axl setup` для пользователя, вызвавшего `sudo`.

Любой Linux: архив tar.gz

Скачайте `axolotl-0.3.0-linux-x64.tar.gz`, распакуйте и запустите `install.sh`. Root не нужен: установка идёт в `~/.local/share/axolotl`, а ссылка на `axl` появляется в `~/.local/bin`:

```
$ tar xzf axolotl-0.3.0-linux-x64.tar.gz
$ sh axolotl-0.3.0/install.sh
```

Другой каталог задаёт `AXL_HOME`: `AXL_HOME=~/tools/axolotl sh axolotl-0.3.0/install.sh`. Если `~/.local/bin` ещё нет в `PATH`, скрипт об этом скажет; добавьте эту строку в `~/.profile` и войдите в систему заново:

```
export PATH="$HOME/.local/bin:$PATH"
```

> Программам, которые через Java открывают окна или играют звук, нужны библиотеки X11 и ALSA системы; `.deb` ставит их сам, при установке из архива это остаётся за вами.

4. Проверка установки

```
$ axl --version
axl 0.3.0
```

`axl --help` выводит список команд, а `axl help <команда>` или `axl <команда> --help` описывает одну из них.

5. Первая программа

Исходники лежат в каталоге исходников, по каталогу на каждую часть имени пакета, как в Java. Создайте каталог проекта и в нём файл `src/app/Main.axl`:

```
package app

import java.util.List

fn greet(name: String?) -> String = "Hello, " + (name?.trim() ?: "stranger")

fn main() {
    val words = List.of(args)
    println(greet(if (words.isEmpty()) null else words.get(0)))
}
```

Запустите его из каталога проекта. Всё, что идёт после имени файла, программа получает в `args`:

```
$ axl run src/app/Main.axl
Hello, stranger
$ axl run src/app/Main.axl Ann
Hello, Ann
```

`axl run` находит каталог исходников по пакету файла, переводит и компилирует все файлы `.axl` и `.java` в нём, проверяет null и запускает `fn main()`. Файлы сохраняйте в UTF-8.

6. Команды

Команда	Что делает
`axl run <file> [args...]`	Компилирует дерево исходников файла и запускает его `fn main()`.
`axl check <dirs...>`	Переводит и компилирует, ничего не сохраняет и печатает ошибки. Быстрая проверка; её же вызывают редакторы.
`axl build -d <dir> <src>`	Переводит и компилирует каталог исходников в class-файлы.
`axl retrace [file]`	Заменяет в старом стеке вызовов строки `.java` на строки `.axl`.
`axl setup`	Устанавливает библиотеки и плагины в локальный Maven-репозиторий.
`axl help [command]`	Описывает команды.

Параметры

Параметр	Команды	Значение
`-cp`, `--classpath`	run, check, build, retrace	Jar-файлы и каталоги классов, которые использует код, через `:`.
`-d <dir>`	build	Куда писать class-файлы.
`-s <dir>`	build	Куда писать сгенерированные `.java` и `.axlmap` (по умолчанию: каталог из `-d`).
`--strict-null`	run, check, build	Считает значения из Java-кода без аннотаций null-безопасности nullable.
`--no-null-checks`	run, check, build	Отключает Error Prone и NullAway: быстрее, но null не проверяется.
`--inline-elvis`	run, check, build	Раскрывает `?:` в операторах без лямбды.
`--no-param-checks`	run, check, build	Убирает проверки null на входе публичных функций и конструкторов для вызовов из Java.
`--no-smap`	build	Оставляет в классах номера строк сгенерированных `.java`-файлов.
`-m`, `--map-dir`	retrace	Каталог со сгенерированными `.java` и `.axlmap`; можно повторять.
`--repository`	setup	Локальный Maven-репозиторий, куда устанавливать.
`-h`, `-V`	все	Справка и версия.

7. Проверка кода: axl check

Добавьте файл, где забыта проверка на null, `src/app/Broken.axl`:

```
package app

fn shout(name: String?) -> String = name.toUpperCase()
```

```
$ axl check src
src/app/Broken.axl:3:42: error: value `name` may be null: use `?.`, `?:` or check `if (name
    != null)` [AXL102]
```

Каждое сообщение имеет вид `файл:строка:столбец: error|warning: текст [код]`. Коды перечислены в `docs/errors.md`: AXL0xx выдают собственные проверки Axolotl, AXL1xx приходят от javac, Error Prone и NullAway и переформулированы в терминах Axolotl. Предупреждения сборку не останавливают.

`axl check`, `axl build` и `axl run` завершаются с кодом 1 при ошибках и 0 без них, поэтому скрипты и CI могут проверять `$?`. `axl check` принимает несколько каталогов исходников, например `axl check src/main src/test`.

Исправьте на `name?.toUpperCase() ?: ""`, и `axl check src` ничего не выведет.

8. Сборка class-файлов: axl build

```
$ axl build -d out/classes -s out/gen src
```

В `out/classes` попадают class-файлы, в `out/gen` сгенерированный Java и карты строк `.axlmap`. Сгенерированный Java это обычный читаемый Java 21. Если код использует другие библиотеки, передайте их через `-cp`, разделяя `:`.

Классы запускаются на любой Java 21 и новее, если на classpath есть библиотека времени выполнения Axolotl. `axl setup` положил её в локальный Maven-репозиторий. Команды `java` в `axl` нет, поэтому для

этого нужна своя Java:

```
$ RT=~/.m2/repository/org/axl/axl-runtime/0.3.0/axl-runtime-0.3.0.jar
$ java -cp "out/classes:$RT" app.Main Bob
Hello, Bob
```

Для чего-то большего, чем несколько файлов, проще сборка Gradle или Maven (раздел 10).

9. Стек вызовов

Классы, собранные `axl build`, сами называют файл `.axl` и строку, поэтому программа, запущенная через `java`, печатает их в каждом стеке вызовов и логе:

```
Exception in thread "main" java.lang.IndexOutOfBoundsException: Index: 5 Size: 1
    ...
    at app.Main.first(Main.axl:5)
    at app.Main.main(Main.axl:8)
```

`axl retrace` нужен для стеков, где ещё указан сгенерированный Java: старых логов или классов, собранных с `--no-smap`. Он читает стек из файла или со стандартного ввода и переводит его на строки `.axl` по файлам `.axlmap` (`-m out/gen`):

```
    at app.Main.first(Main.java:9)
    at app.Main.main(Main.java:12)
```

```
$ axl retrace -m out/gen trace.txt
    at app.Main.first(Main.axl:5)
    at app.Main.main(Main.axl:8)
```

10. Проекты Gradle и Maven

После `axl setup` обе системы сборки находят Axolotl в локальном Maven-репозитории. Error Prone, NullAway и прочие зависимости они по-прежнему скачивают из Maven Central, и им нужен JDK 21.

Gradle

```
// settings.gradle.kts
pluginManagement { repositories { mavenLocal(); gradlePluginPortal() } }
dependencyResolutionManagement { repositories { mavenLocal(); mavenCentral() } }

// build.gradle.kts
plugins {
    application
    id("org.axl.lang") version "0.3.0"
}
application { mainClass = "app.Main" }
```

Исходники кладутся в `src/main/axl` и `src/test/axl`; `gradle run` собирает и запускает программу.

Maven

Новый проект из архетипа, готовый к сборке через `mvn package`:

```
$ mvn archetype:generate \
    -DarchetypeGroupId=org.axl -DarchetypeArtifactId=axl-archetype-quickstart \
    -DarchetypeVersion=0.3.0 \
    -DgroupId=org.example -DartifactId=demo -Dpackage=org.example.demo
```

11. Обновление и удаление

Установите новый `.deb` через `sudo apt install ./...deb` или запустите `install.sh` из нового архива: он заменит прежнюю установку.

- Пакет `.deb`: `sudo apt remove axolotl`.
- Архив: удалите `~/.local/share/axolotl` (или ваш `AXL_HOME`) и `~/.local/bin/axl`.

Библиотеки, установленные `axl setup`, остаются в локальном Maven-репозитории в каталоге `org/axl`; удалите и его, если они не нужны ни одному проекту.

12. Решение проблем

Проблема	Что делать
E: Unable to locate package от `apt`.	Укажите путь с `./` (или полный путь): `sudo apt install ./axolotl-...deb`.
`apt` не может удовлетворить зависимости на старом выпуске.	Используйте архив tar.gz: у него нет зависимостей.
axl: command not found после установки из архива.	Добавьте `~/.local/bin` в `PATH` (раздел 3) и откройте новый терминал.
Maven или Gradle не находит `org.axl.lang` или `axl-maven-plugin`.	Выполните `axl setup` от имени пользователя, который собирает проект; проверьте, что в сборке есть `mavenLocal()`. Если в `MAVEN_OPTS` задан `-Dmaven.repo.local`, повторите `axl setup` из этой оболочки или укажите `axl setup --repository <каталог>`.
`java` не найдена или старее 21 при запуске собранных классов или Gradle.	Установите JDK 21; собственная среда `axl` служит только ему.
Первая компиляция идёт долго.	`axl` при каждом вызове запускает JVM и загружает javac и NullAway; несколько секунд это нормально. Для быстрых экспериментов `--no-null-checks` пропускает NullAway.

Дальше

Документация по языку, плагинам Gradle и Maven и редакторам: https://github.com/succubus-uwu/axolotl/tree/v0.3.0/docs. Релизы и установщики: https://github.com/succubus-uwu/axolotl/releases.